Contents

AeonVoice Engine And Config Reference

This document explains how AeonVoice is initialized, where configuration is loaded from, which settings are available, and how those settings influence synthesis.

It is based on the current code in:

- `src/core/engine.cpp`
- `src/core/params.cpp`
- `src/include/core/engine.hpp`
- `src/include/core/params.hpp`
- `src/include/core/quality_setting.hpp`

- src/core/document.cpp
- src/core/hts_label.cpp
- src/include/AeonVoice.h
- dotnet/AeonVoice/AeonVoiceEngine.cs

Runtime Overview

At a high level, AeonVoice works like this:

1. An engine is created.
2. The engine discovers language and voice resources from the configured data paths.
3. The engine registers global, language-specific, and voice-specific settings.
4. The engine loads AeonVoice.conf once at startup.
5. Client code creates a message with text plus synthesis parameters.
6. A document and one or more utterance objects are built from the text.
7. Effective rate, pitch, volume, quality, punctuation, and voice settings are resolved.
8. HTS synthesis runs and audio is streamed back through callbacks or the wrapper API.

Where Config Comes From

Native engine defaults

The native engine initializes default paths in engine::init_params:

- data_path defaults to the compile-time DATA_PATH
- config_path defaults to the compile-time CONFIG_PATH

These can be overridden with environment variables:

- AEONVOICE_DATA_PATH
- AEONVOICE_CONFIG_PATH

The logic is in src/core/engine.cpp.

Native config file name

At engine startup, AeonVoice looks for:

- Windows: AeonVoice.ini
- Other platforms: AeonVoice.conf

The config is loaded once during engine construction.

.NET wrapper behavior

The .NET wrapper in dotnet/AeonVoice/AeonVoiceEngine.cs tries these packaged paths first:

- ./aeonvoice/data
- ./aeonvoice/config

If present, those paths are passed into the native engine. Otherwise the native engine falls back to its normal path resolution.

Config Scope

Settings can be applied at three levels:

Global

```
default_rate=1.0
quality=standard
```

Language-specific

```
languages.english.default_rate=1.1
languages.russian.use_pseudo_english=false
```

Voice-specific

```
voices.leena.default_pitch=0.96
voices.elena.enabled=false
```

The engine registers:

- global voice/text/verbosity settings
- language settings
- voice settings

Then language settings inherit from global settings, and voices inherit from their language/global settings.

Available Settings

Rate, pitch, and volume

These are defined in `src/core/params.cpp` and documented in `doc/Configuration.md`.

`1.0` means neutral behavior.

Setting	Default	Min	Max	Notes
`default_rate`	1	0.2	5	Effective default is constrained by `min_rate`/`max_rate`
`min_rate`	0.5	0.2	1	Lower bound for rate
`max_rate`	2	1	5	Upper bound for rate
`default_pitch`	1	0.5	2	Effective default is constrained by `min_pitch`/`max_pitch`
`min_pitch`	0.5	0.5	1	Lower bound for pitch
`max_pitch`	2	1	2	Upper bound for pitch
`default_volume`	1	0.25	4	Effective default is constrained by `min_volume`/`max_volume`
`min_volume`	0.25	0.25	1	Lower bound for volume
`max_volume`	2	1	4	Upper bound for volume
`cap_pitch_factor`	1.3	0.5	2	Used only when capitals are indicated by pitch
`min_sonic_rate`	1	0.2	6	Only relevant in builds with Sonic enabled

These settings can be applied globally, per language, or per voice.

Quality

`quality` is defined in `src/include/core/quality_setting.hpp`.

Accepted values:

- `min`, `minimum`, `0`
- `standard`, `std`, `default`, `50`
- `max`, `maximum`, `100`

Behavior:

Value	Sample rate	Notes
`min`	16 kHz	fastest/lowest quality
`standard`	24 kHz	default
`max`	24 kHz	highest quality, slower startup

Punctuation

`punctuation_mode` accepted values:

- `none`
- `some`
- `all`

`punctuation_list` is a character set used when `punctuation_mode=some`.

Defaults from code:

- `punctuation_mode=none`
- `punctuation_list=+=<>~@#$%^&*|`

Capital letters

`indicate_capitals` accepted values:

- off: `off`, `no`, `none`, `false`
- `word`
- `pitch`
- `sound`

Default:

- `indicate_capitals=no`

Related setting:

- `cap_pitch_factor`

Language switching and voice profiles

`voice_profiles` is a comma-separated list of profiles:

```
voice_profiles=Leena,Leena+Alan,Aleksandr+CLB
```

`prefer_primary_language` is a boolean:

- default `true`

The first voice in a profile is the primary voice.

Text and language behavior

`stress_marker`

- default `+`
- applies to Russian and Ukrainian stress marking behavior

`enable_bilingual`

- default `true`

`languages.<language>.use_pseudo_english`

- default `true` for supported languages

`languages.esperanto.present_as_english`

- default `false`

Resource enable/disable

`languages.<language>.enabled`

- boolean
- default `true`

`voices.<voice>.enabled`

- boolean
- default `true`

Stream settings

These are code-supported settings not shown in the sample config:

Setting	Default	Min	Max
`stream.fixed_size`	1	1	10
`stream.view_size`	3	1	10

Boolean Values

Per `doc/Configuration.md`, boolean settings accept:

True:

- `true`
- `yes`
- `on`
- `1`

False:

- `false`
- `no`
- `off`
- `0`

How Config Affects Synthesis

The main flow is:

1. Engine-level config is registered on startup.
2. The config file is loaded into the engine.
3. A client message provides synthesis parameters such as:
    - voice profile
    - absolute rate/pitch/volume
    - relative rate/pitch/volume
    - punctuation mode
    - capitals mode
4. `document.cpp` converts those message-level parameters into `utterance` settings.
5. `hts_label.cpp` computes effective rate, pitch, and volume using:
    - absolute client override
    - relative client multiplier

- configured defaults/min/max

This means the config file shapes the baseline behavior, and client code can still steer each utterance.

C API Usage

The public C API is declared in `src/include/AeonVoice.h`.

Important structs:

- `AeonVoice_init_params`
- `AeonVoice_callbacks`
- `AeonVoice_synth_params`

Important fields in `AeonVoice_synth_params`:

- `voice_profile`
- `absolute_rate`, `absolute_pitch`, `absolute_volume`
- `relative_rate`, `relative_pitch`, `relative_volume`
- `punctuation_mode`
- `punctuation_list`
- `capitals_mode`

Absolute values use normalized range `-1..1`.

Relative values are multipliers, and `1` means neutral.

.NET Usage

The managed wrapper exposes:

- `AeonVoiceEngine`
- `SynthesizeToPcm16(text, voiceProfile)`
- `SynthesizeToPcm16(text, voiceProfile, SynthesisOptions)`

`SynthesisOptions` currently exposes:

- `RelativeRate`
- `RelativePitch`
- `RelativeVolume`

These are passed through to the native `relative_*` synthesis parameters.

Minimal Examples

Global tuning

```
quality=standard
default_rate=0.98
default_pitch=1.0
default_volume=1.0
```

Voice-specific tuning

```
voices.leena.default_rate=0.92
voices.leena.default_pitch=0.96
voices.leena.default_volume=1.05
```

Disable a voice

```
voices.elena.enabled=false
```

Selective punctuation

```
punctuation_mode=some
punctuation_list=@$/\
```

Practical Reading Order

If you want to understand the system quickly, read in this order:

1. `doc/Configuration.md`
2. `src/core/engine.cpp`
3. `src/core/params.cpp`
4. `src/core/document.cpp`
5. `src/core/hts_label.cpp`
6. `src/include/AeonVoice.h`
7. `dotnet/AeonVoice/AeonVoiceEngine.cs`

That path goes from "what users can set" to "how those settings change audio."

Regenerating Compiled Voice Data

If the problem is in the voice model rather than wrapper/config tuning, the relevant process is not editing `AeonVoice.conf`. It is regenerating the compiled voice artifacts under `data/voices/<voice>/`.

This workflow is described in `doc/CustomVoice.md`. The short version is:

1. Build AeonVoice helpers in dev mode

The training flow expects repo-built helper binaries:

```
scons dev=True -j4
```

This produces:

- `local/bin/AeonVoice-make-hts-labels`
- `local/bin/AeonVoice-transcribe-sentences`

These are used during label preparation and training support.

2. Prepare the external training toolchain

The repo does not contain the full HTS training toolchain. Training depends on external binaries such as:

- HTK tools: `HLEd`, `HVite`
- HTS tools: `HHEd`
- SPTK tools: `mcep`, `pitch`
- Festival
- Praat

The helper scripts and expected config fields are documented in `doc/CustomVoice.md`.

Useful repo-side helpers:

- `src/scripts/general/setup_environment_debian`
- `src/scripts/general/check_training_env.sh`
- `src/scripts/general/training.cfg`

3. Prepare recordings and transcripts

You need:

- a wave directory

- training text
- test text
- a speaker/voice identifier
- the target language
- an output directory

The guide recommends a consistent mono PCM corpus and a held-out evaluation set.

4. Initialize a training workspace

The repo includes an HTS demo scaffold plus wrapper utilities.

Example from the guide:

```
mkdir -p work/henry-train
cd work/henry-train
python3 ../../src/scripts/general/voice-building-utils init
```

This creates a working HTS-style training workspace.

5. Generate labels and linguistic features

AeonVoice needs HTS labels aligned to the language logic used by the engine.

The repo helper is:

- `src/utils/make-hts-labels.cpp`

The guided command is:

```
python3 ../../src/scripts/general/voice-building-utils label
```

This is the step where text/transcript data becomes synthesis-ready labels.

6. Run HTS training and export

Once the workspace is prepared:

```
./configure
make
```

Or use staged helper subcommands such as:

- `segment`
- `extract-f0`
- `make-questions`
- `realign`
- `export-voice`

These are mentioned in `doc/CustomVoice.md`.

7. Produce AeonVoice runtime artifacts

AeonVoice does not consume raw recordings directly. It consumes compiled voice artifacts in a specific directory layout.

For each sample-rate directory, usually `16000/` and/or `24000/`, the runtime expects files like:

- `voice.data`
- `dur.pdf`
- `mgc.pdf`
- `lf0.pdf`
- `bap.pdf`

- `mgc.win1`, `mgc.win2`, `mgc.win3`
- `lf0.win1`, `lf0.win2`, `lf0.win3`
- `bap.win1`, `bap.win2`, `bap.win3`
- `tree-dur.inf`, `tree-mgc.inf`, `tree-lf0.inf`, `tree-bap.inf`
- `bpf.txt`

These are the compiled voice data files that AeonVoice loads during synthesis.

8. Place the new voice into repo layout

The resulting voice goes under:

```
data/voices/<voice_id>/
  voice.info
  voice.params
  16000/
  24000/
```

Important metadata files:

`voice.info`

```
name=HenryWarm
language=English
gender=male
format=4
revision=1
```

`voice.params`

```
beta=0.4
gain=1.0
key=173
```

9. Validate in AeonVoice runtime

After replacing or adding compiled voice data, test with the native runtime:

```
export AEONVOICE_DATA_PATH="$(pwd)/data"
export LD_LIBRARY_PATH="$(pwd)/build/linux/core:$(pwd)/build/linux/audio:$(pwd)/build/linux/lib"

echo "Hello from AeonVoice" | build/linux/test/AeonVoice-test -p Leena -o /tmp/sample.wav
```

10. Practical interpretation

If you are asking "how do I regenerate compiled voice data for Leena?", the repo's answer is:

1. Gather or improve the underlying recordings and transcripts.
2. Re-run the HTS training/export workflow using the helper scripts.
3. Replace the compiled artifacts in `data/voices/leena/<sample-rate>/`.
4. Re-test with the runtime and listening samples.

That is the model-level path. Editing `AeonVoice.conf` only changes how AeonVoice uses an already compiled voice model.